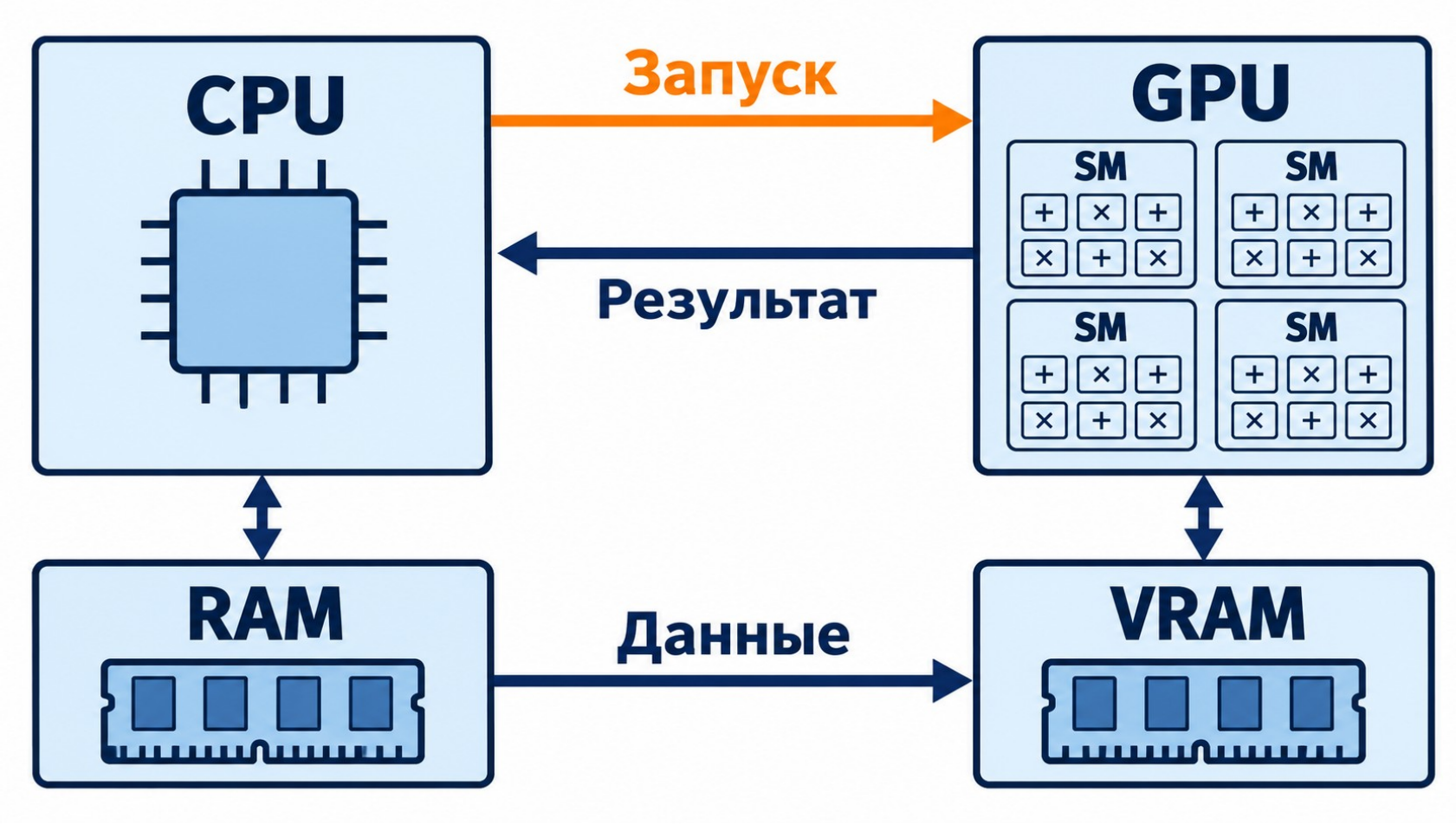
NVIDIA GPU и CUDA



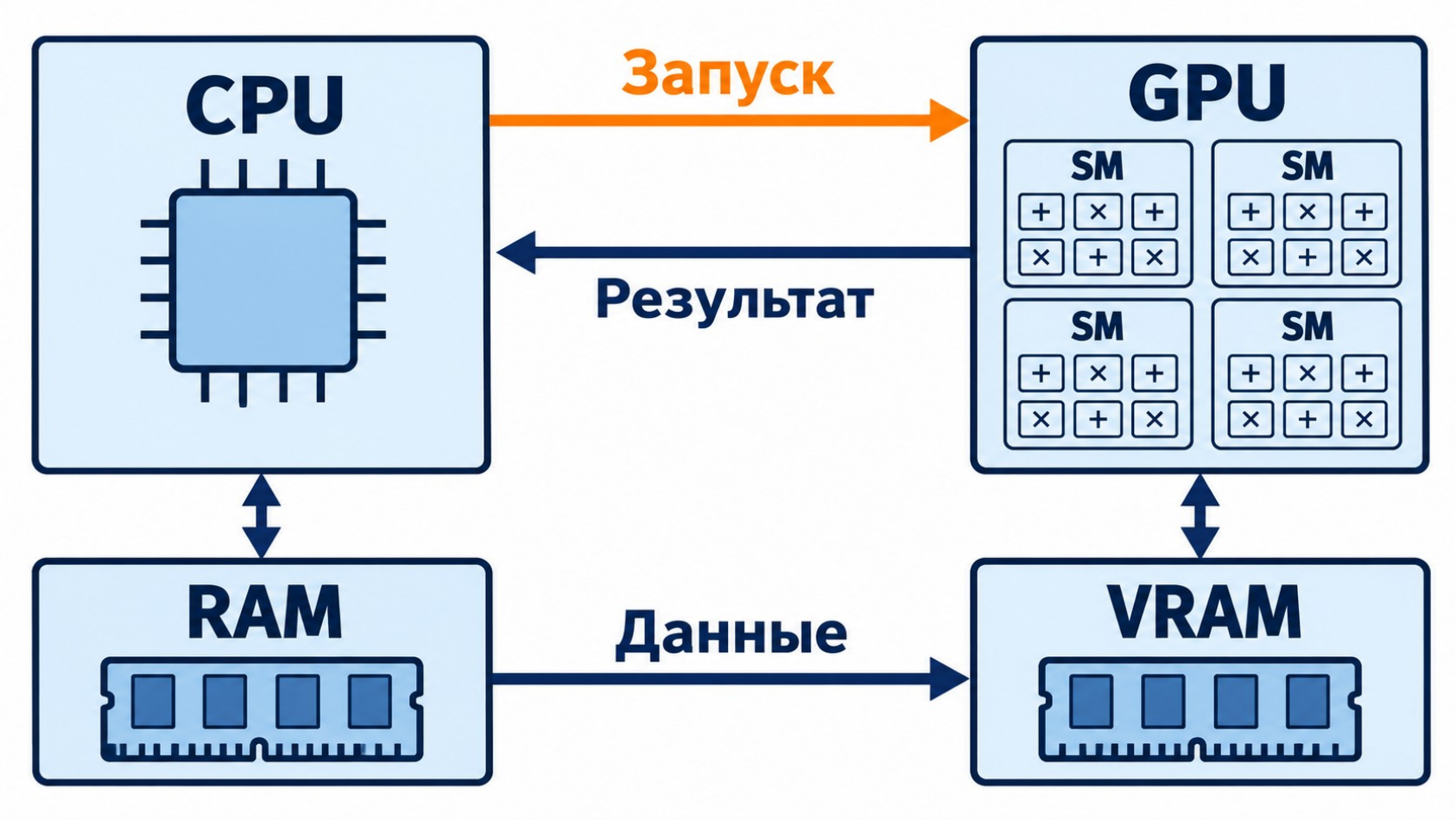
Устройство, выполнение и диагностика ML-вычислений.

NVIDIA

Сегодня на занятии

- Устройство GPU и выполнение CUDA
- Память и передача данных
- Точность и матричные операции
- Измерение и мониторинг

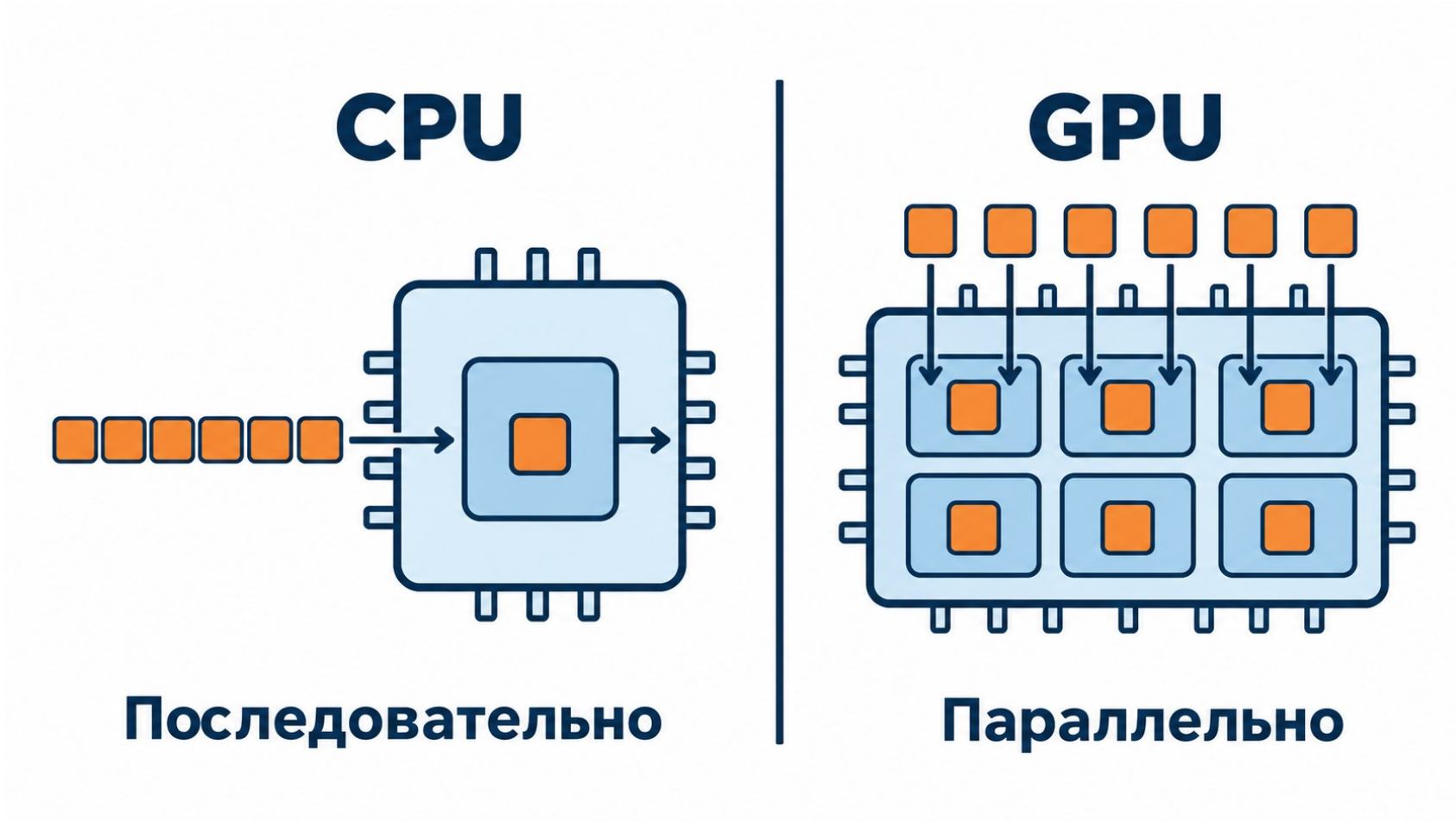
Путь ML-запроса



Подготовка → передача → вычисление → ответ.

NVIDIA

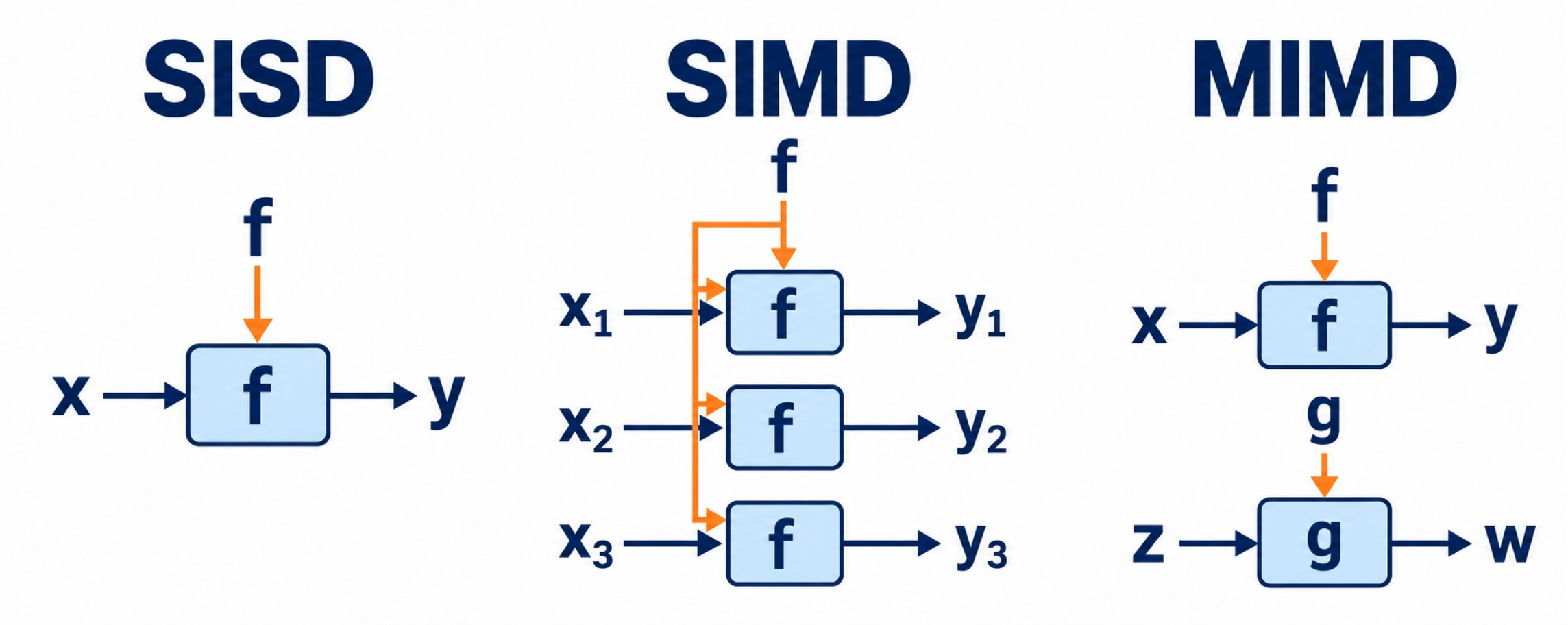
CPU и GPU



Большой объём независимой работы помогает загрузить GPU.

NVIDIA

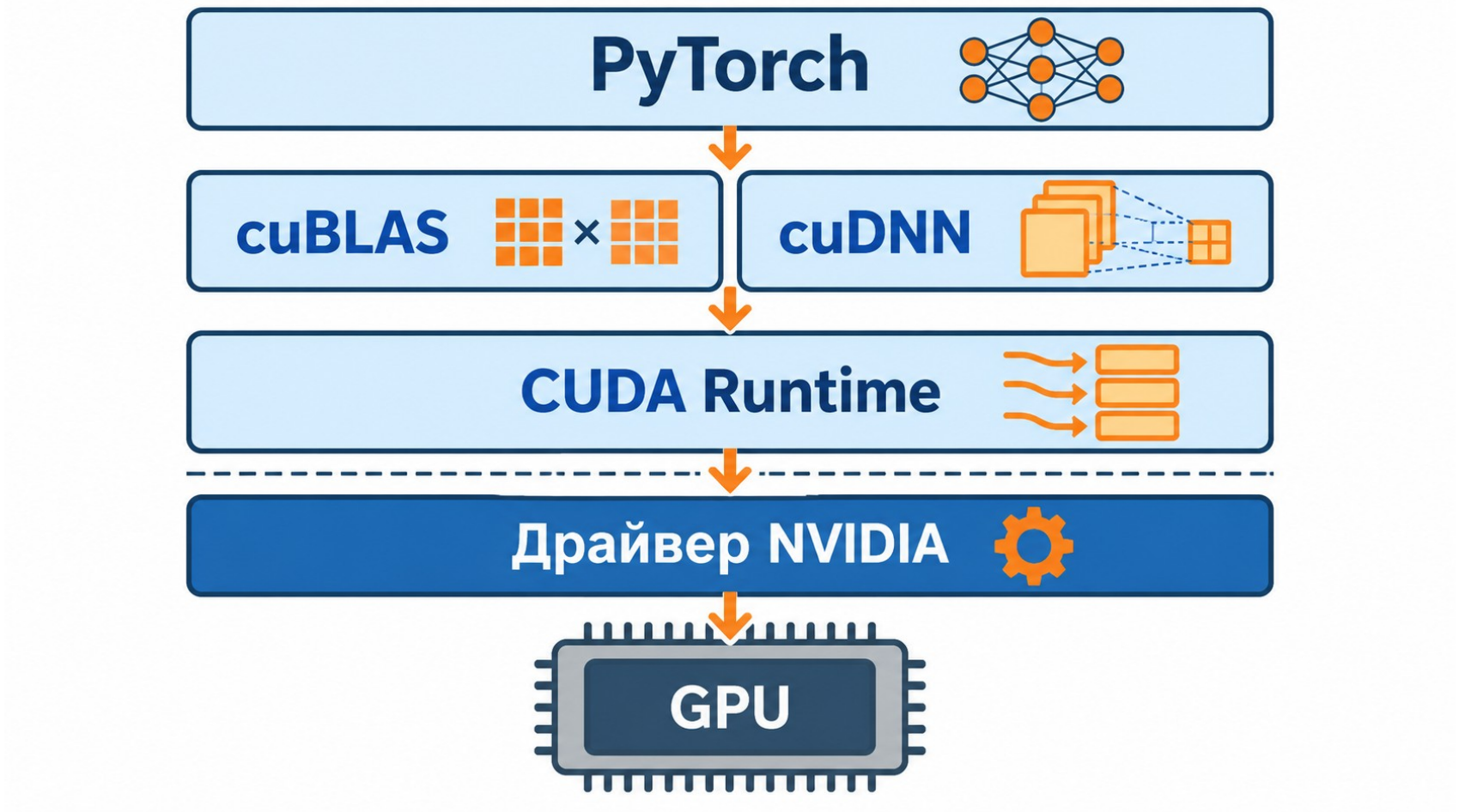
SISD, SIMD и MIMD



Команды и данные.

NVIDIA

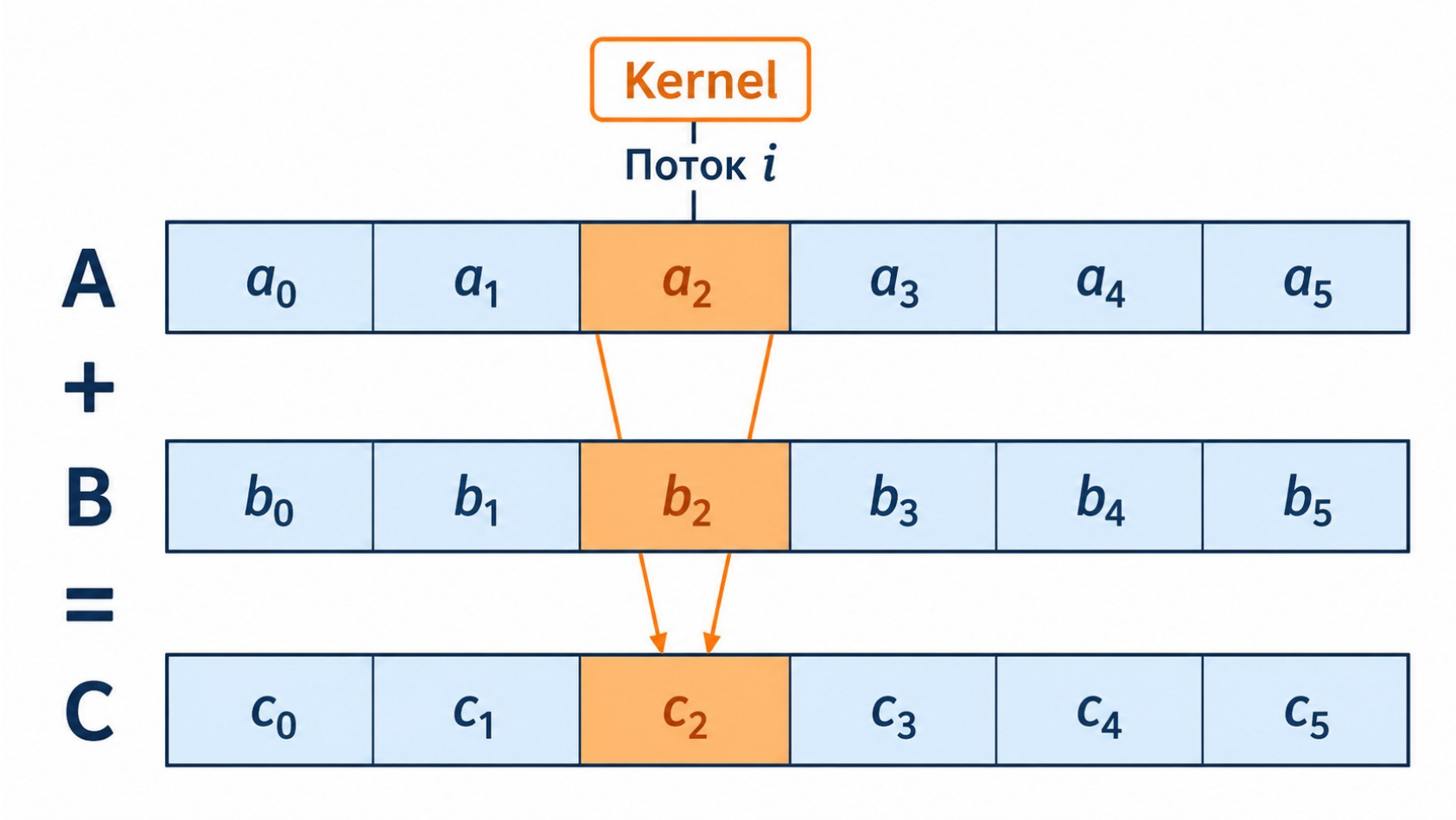
Что такое CUDA



Фреймворк вызывает библиотеки и runtime; драйвер управляет устройством.

NVIDIA

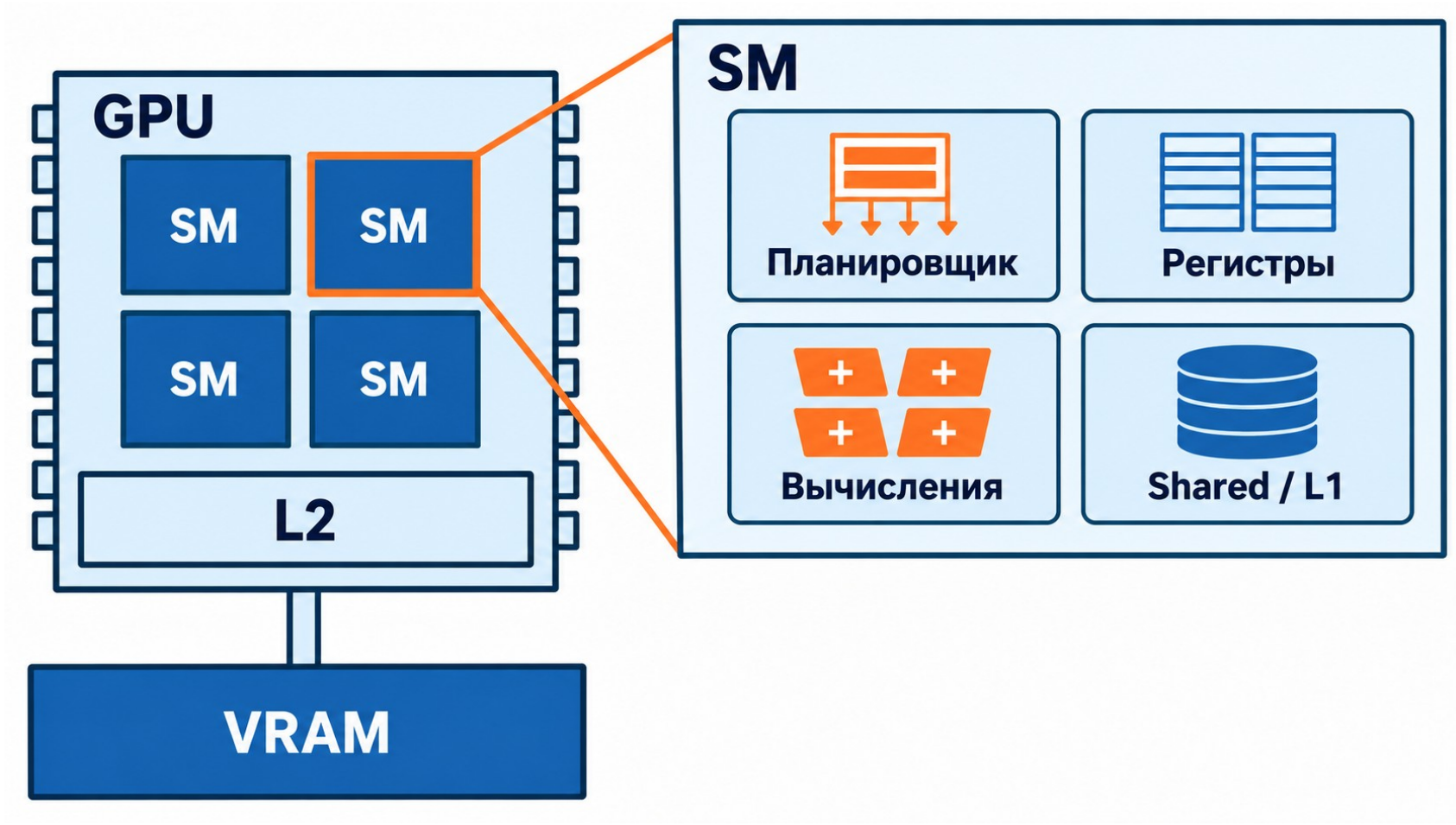
Kernel: одна операция, много потоков



Каждый поток вычисляет свой элемент результата.

NVIDIA

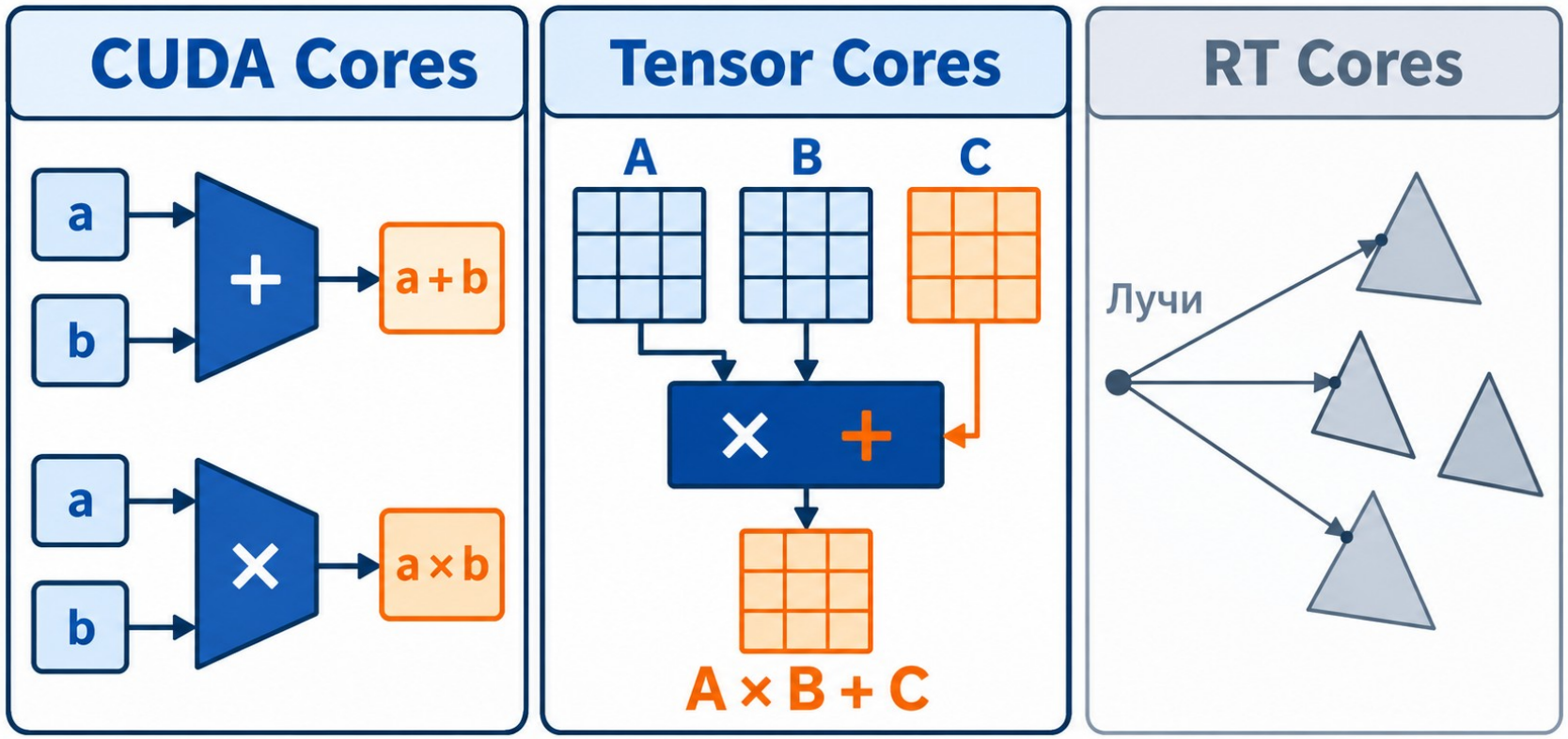
GPU и Streaming Multiprocessor



SM объединяет планирование, вычисления и локальную память.

NVIDIA

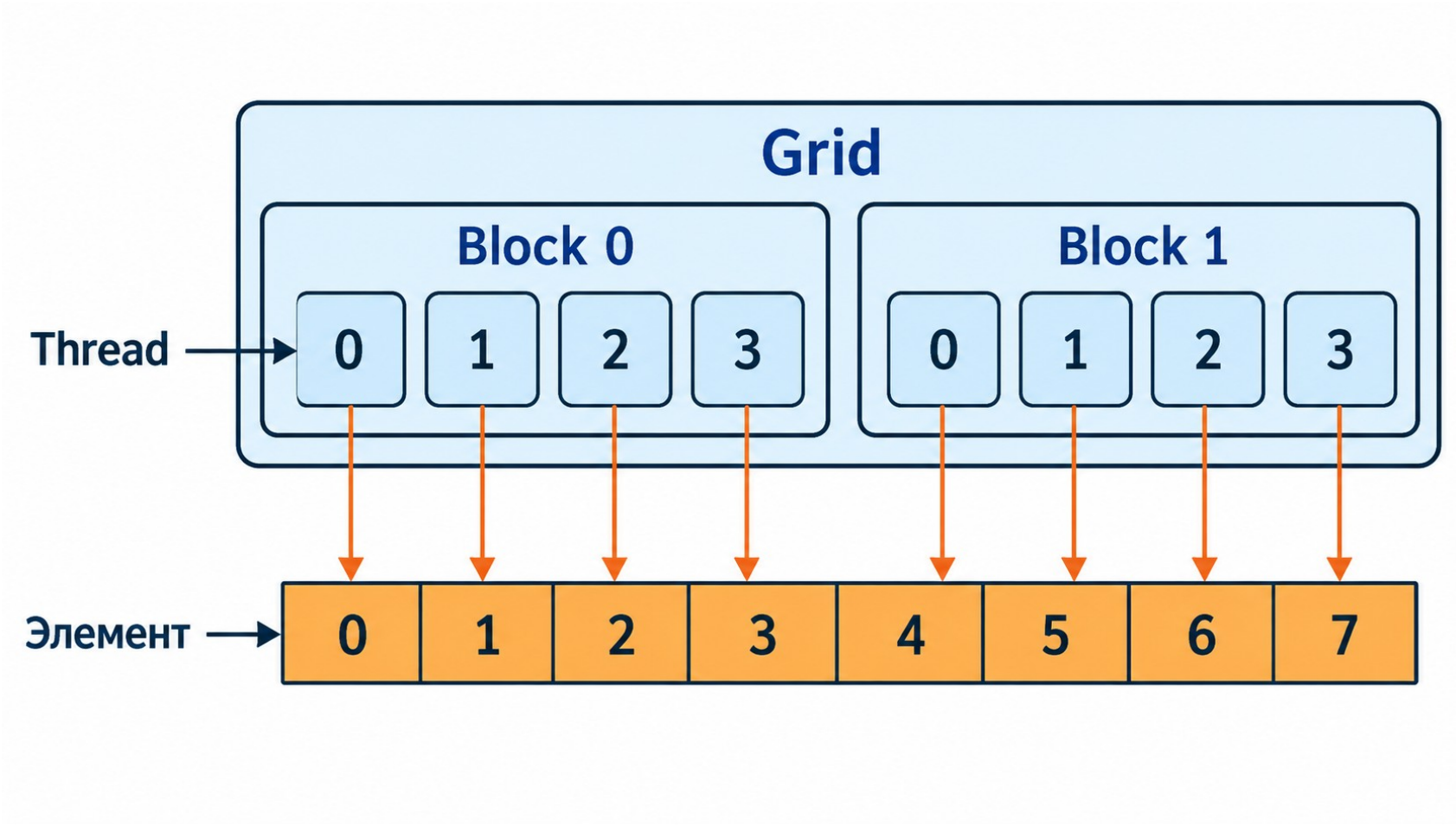
CUDA, Tensor и RT Cores



Разные вычислительные блоки решают разные задачи.

NVIDIA

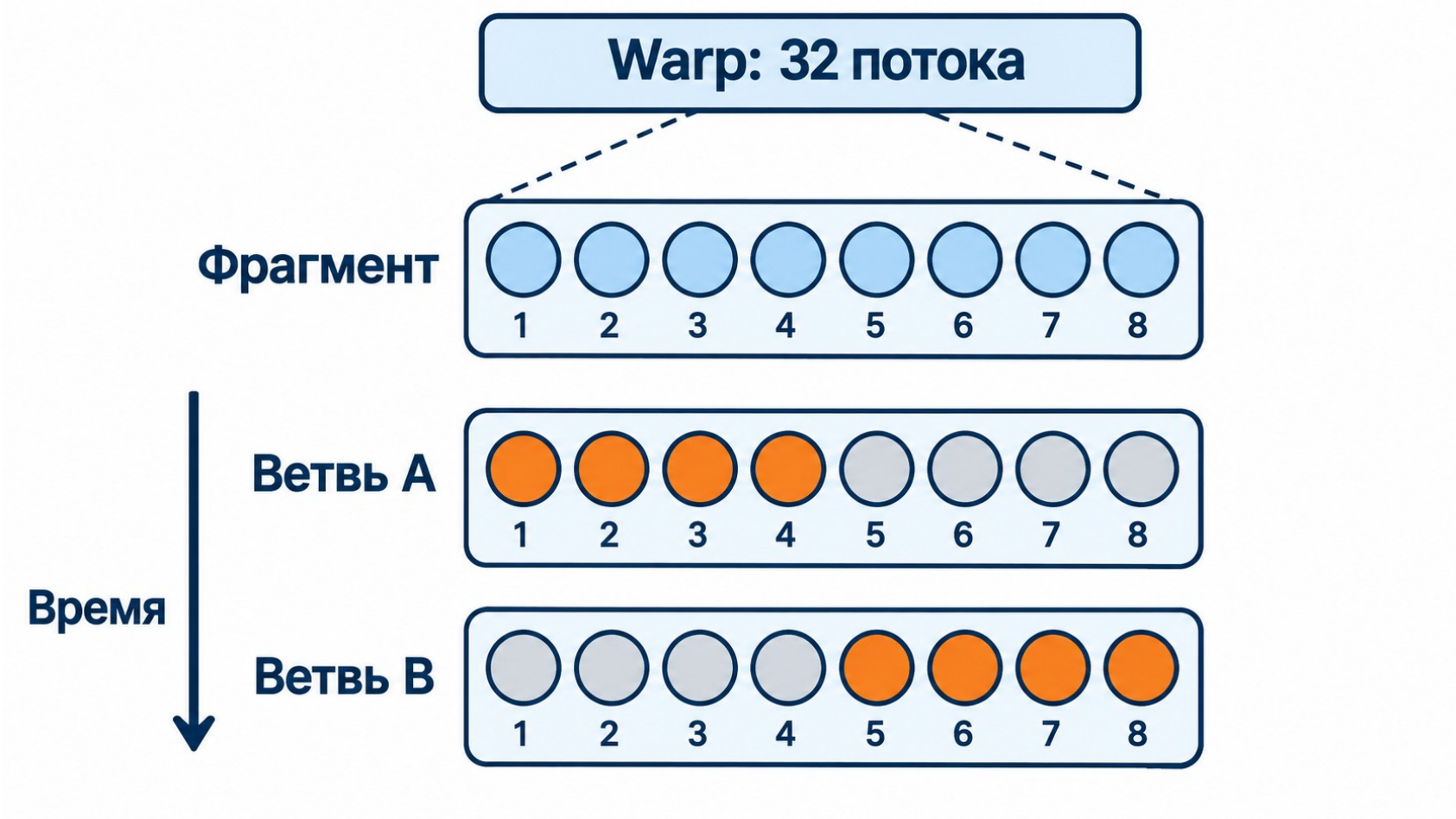
Thread, block и grid



Поток обрабатывает свой элемент.

NVIDIA

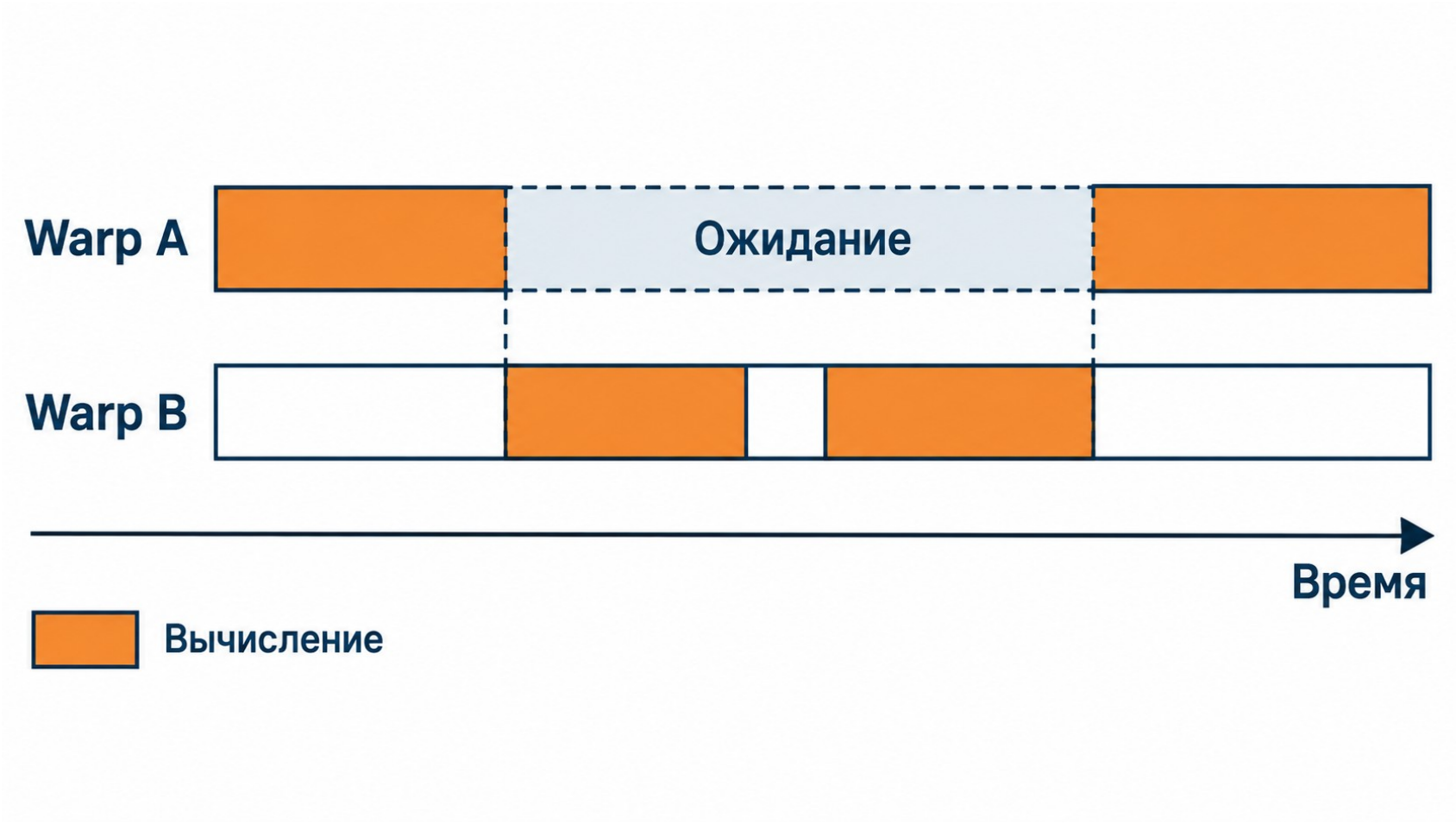
Warp и ветвления



Разные ветви используют разные маски активных потоков.

NVIDIA

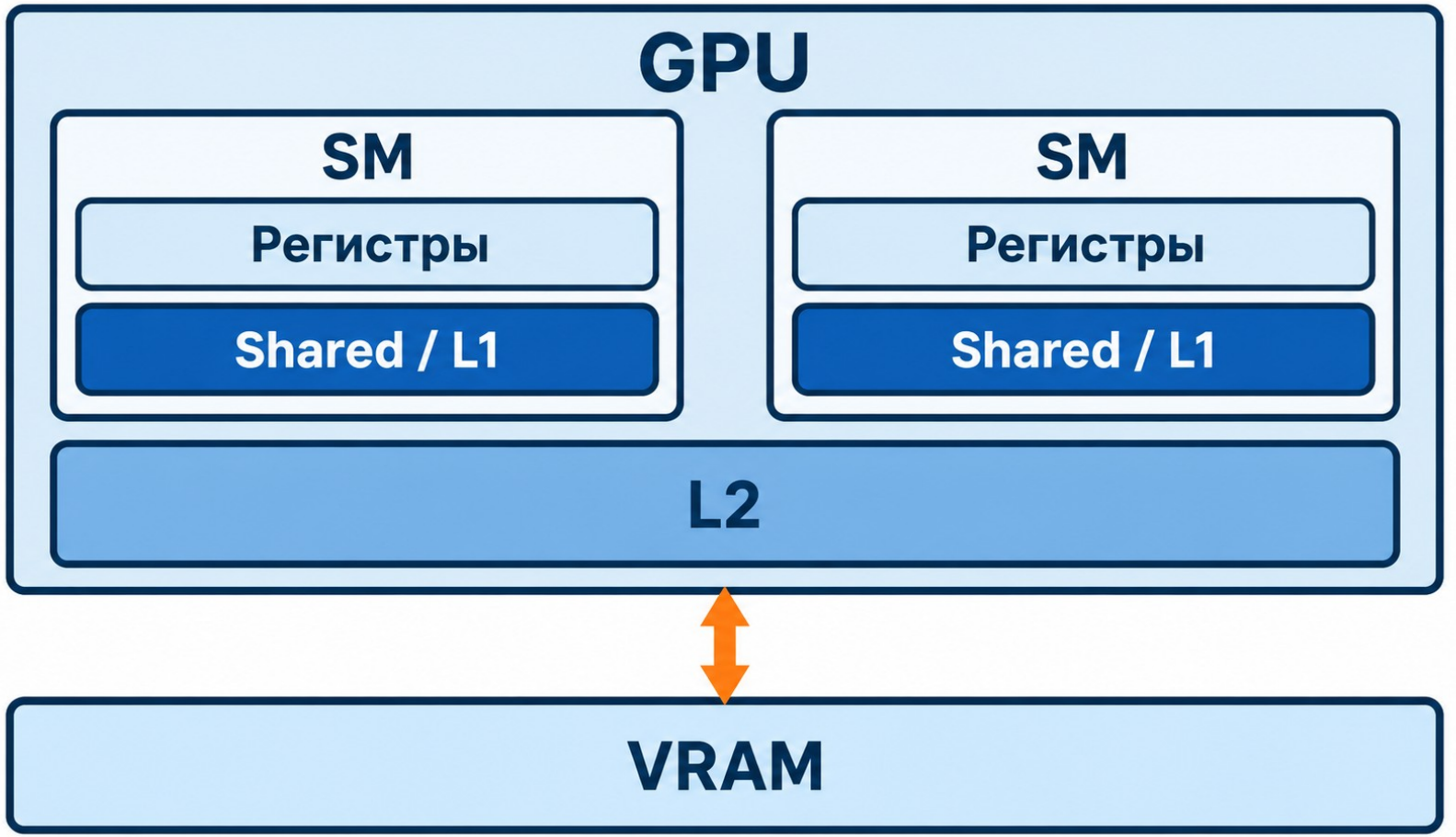
Планирование и ожидание данных



Пока один warp ждёт данные, другой может выполнять инструкции.

NVIDIA

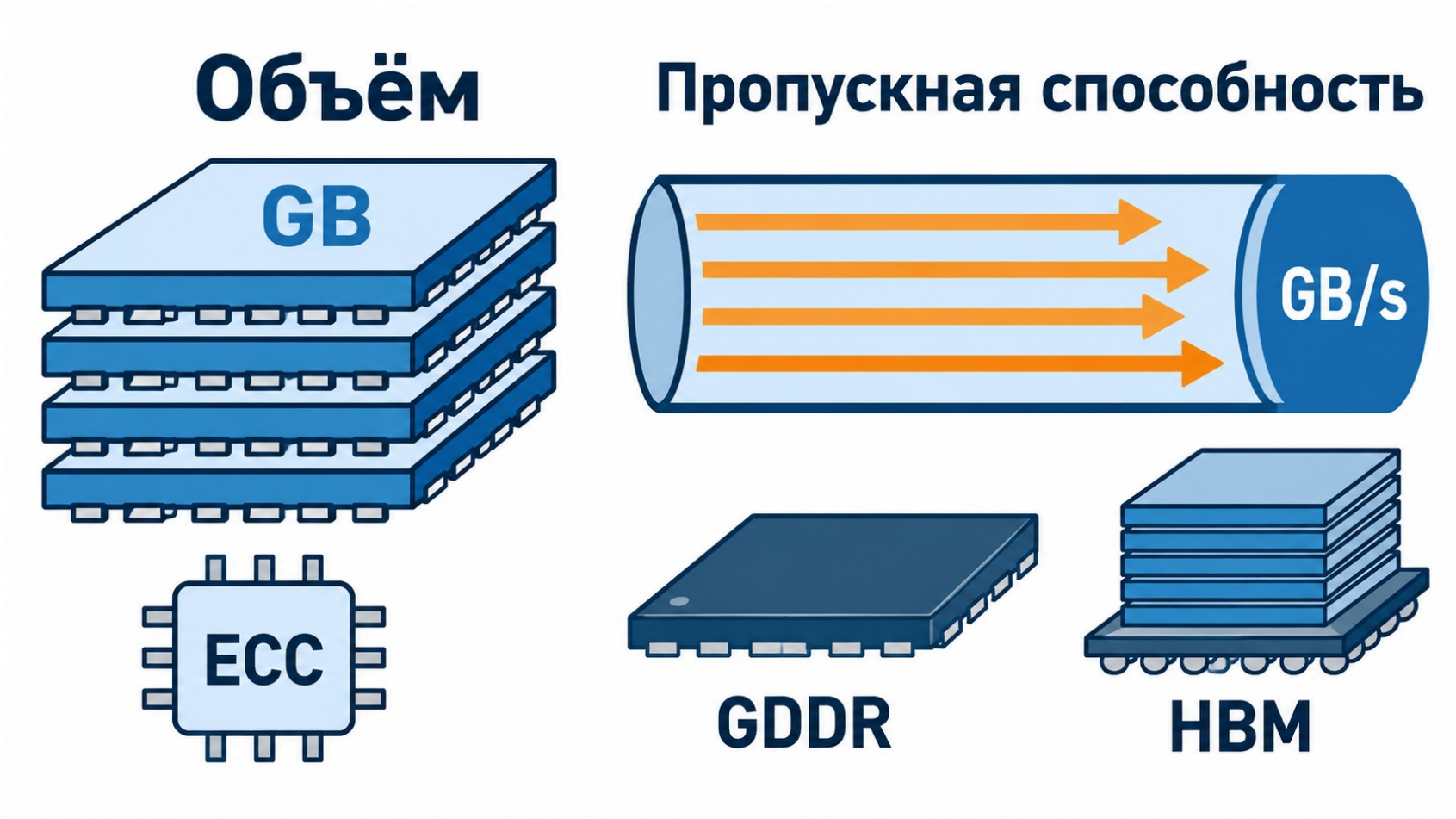
Иерархия памяти



Повторное использование близких данных сокращает обращения к VRAM.

NVIDIA

Видеопамять: объём и скорость



Ёмкость, пропускная способность и ECC отвечают на разные вопросы.

NVIDIA

PCIe и NVLink

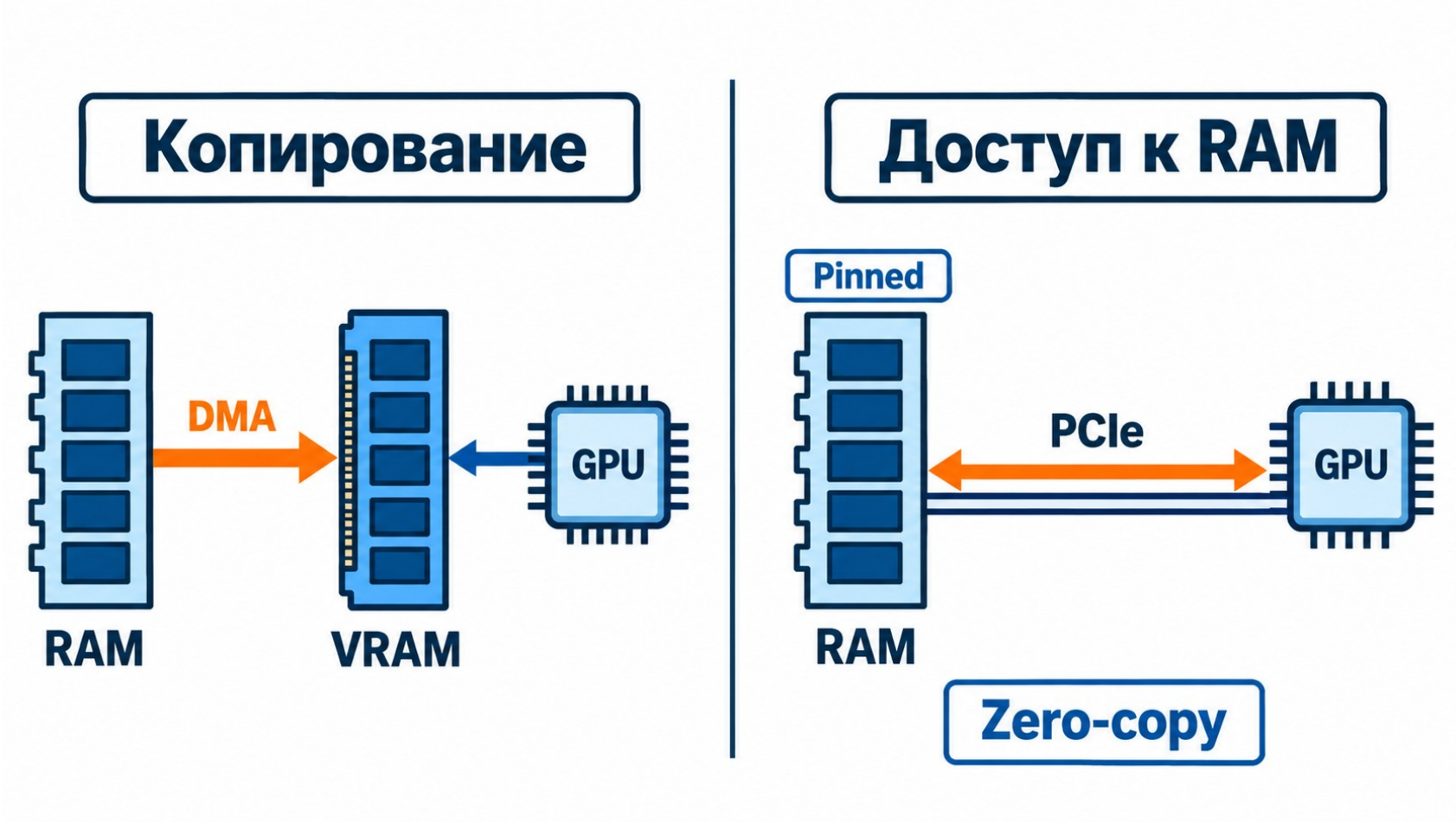
Интерфейс	→, GB/s	↔, GB/s	Суммирование
PCIe 3.0 x16	≈15,8	≈31,5	16 линий
PCIe 4.0 x16	≈31,5	≈63	16 линий
PCIe 5.0 x16	≈63	≈126	16 линий
NVLink 3 · A100 SXM	300	600	Все NVLink одной GPU
NVLink 4 · H100 SXM	450	900	Все NVLink одной GPU
NVLink 5 · Blackwell	900	1800	Все NVLink одной GPU

Теоретические пределы; скорость конкретного пути зависит от топологии.

→ Одно направление ↔ Приём + передача

NVIDIA

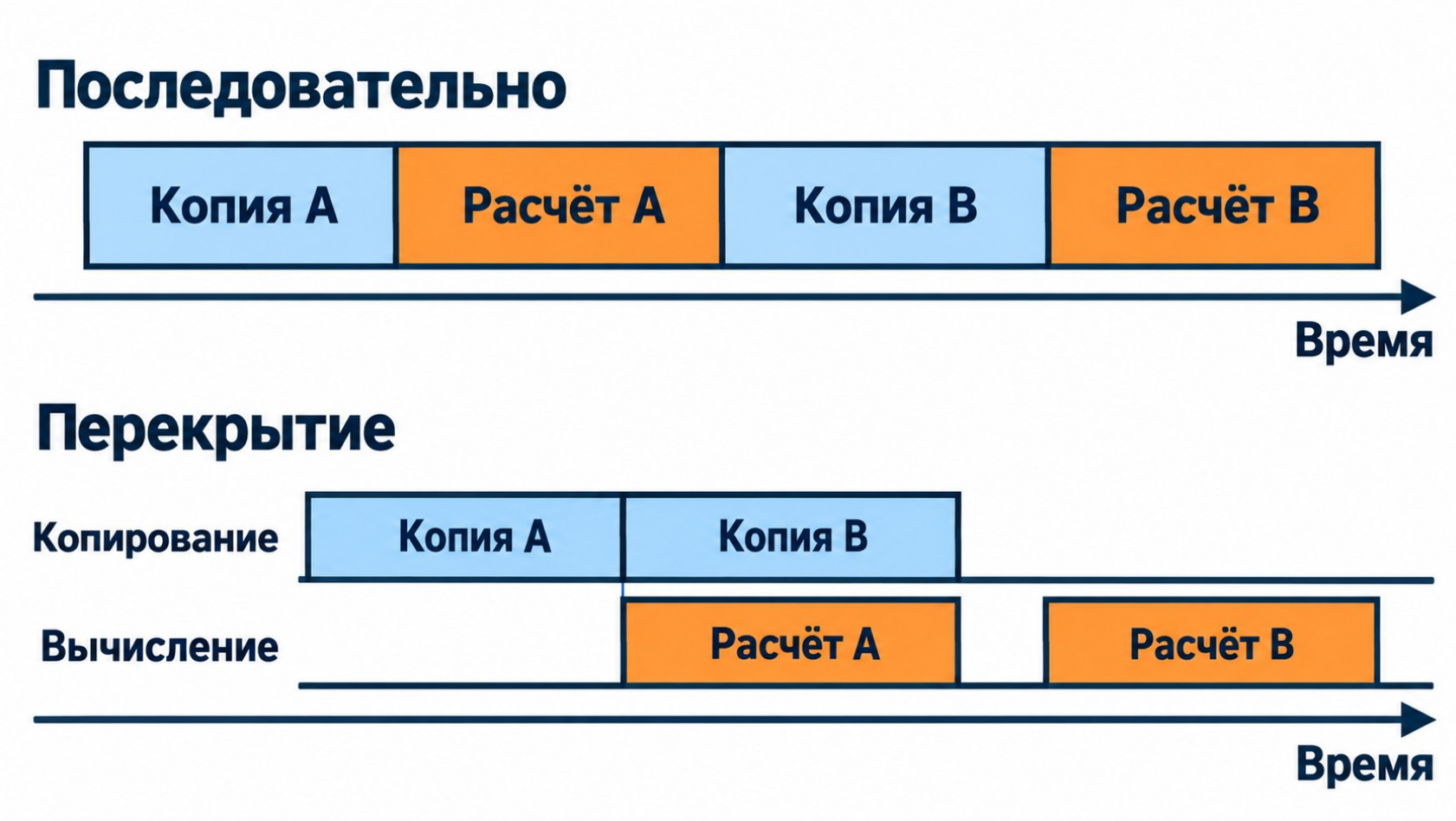
DMA и zero-copy



Способ передачи выбирают с учётом повторных обращений к данным.

NVIDIA

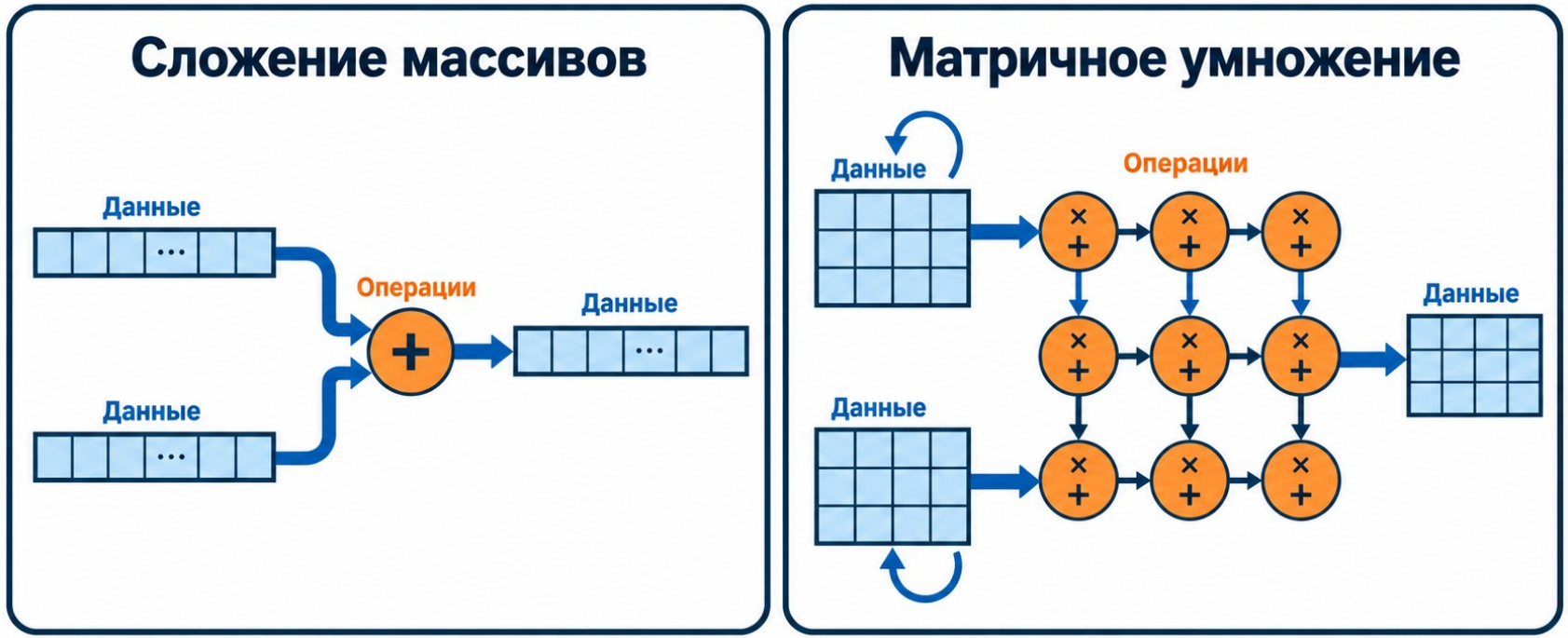
Асинхронность и CUDA streams



Перекрытие передачи и вычисления возможно при выполнении условий.

NVIDIA

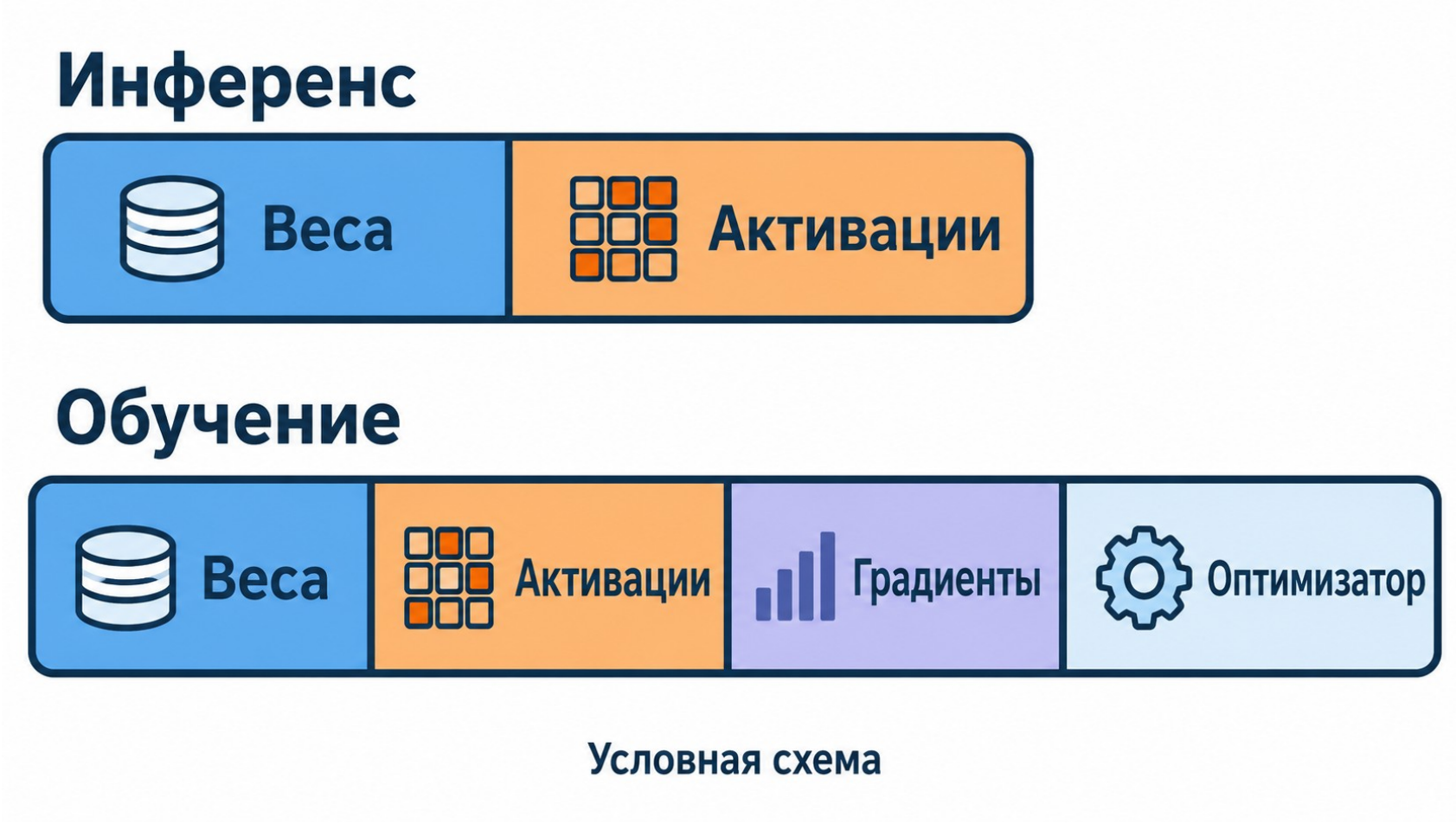
Вычисления или память



Количество операций на переданный байт меняет характер ограничения.

NVIDIA

Куда расходуется VRAM

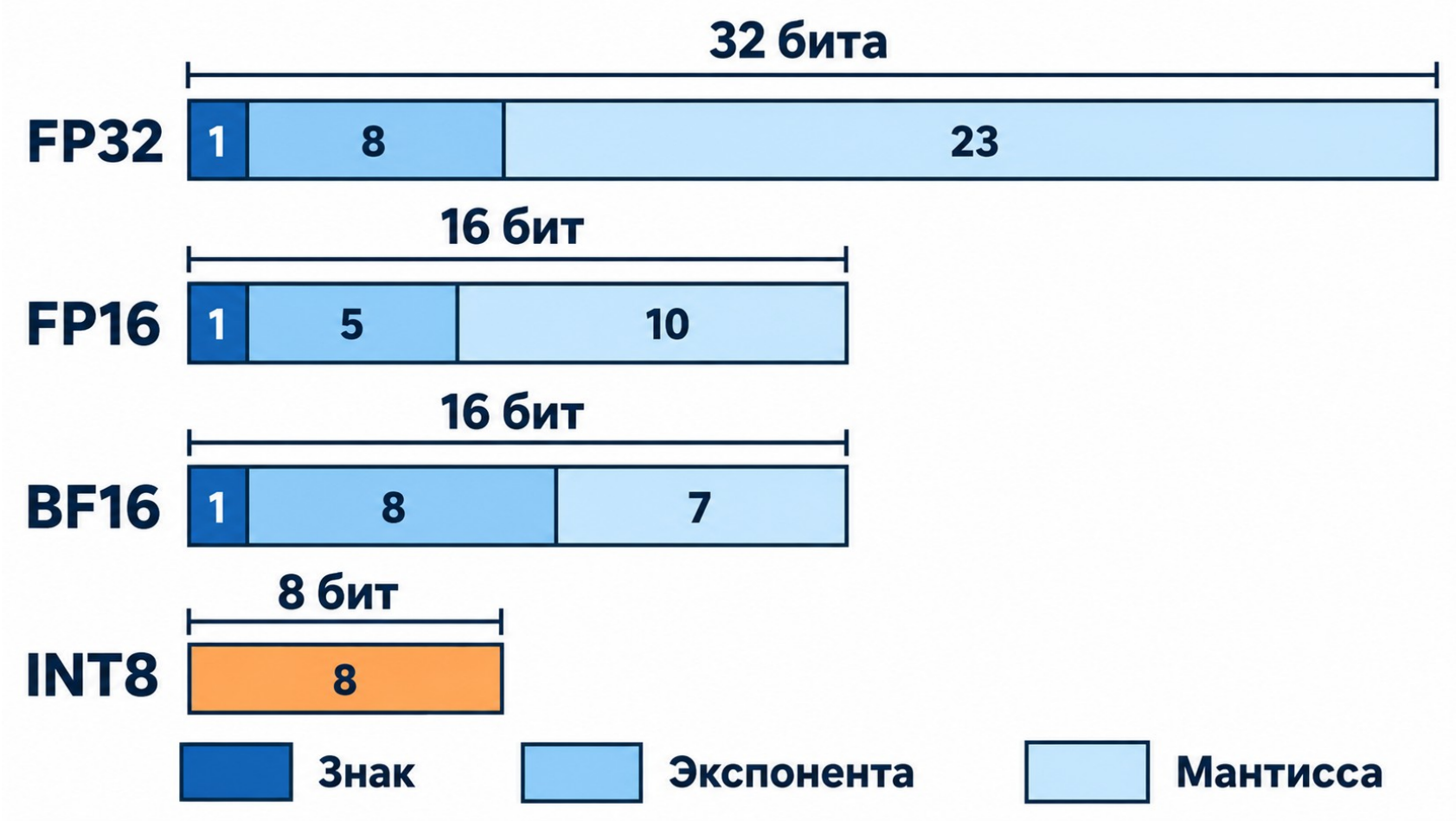


Условная схема

Обучение хранит больше состояния, чем одни веса модели.

PyTorch

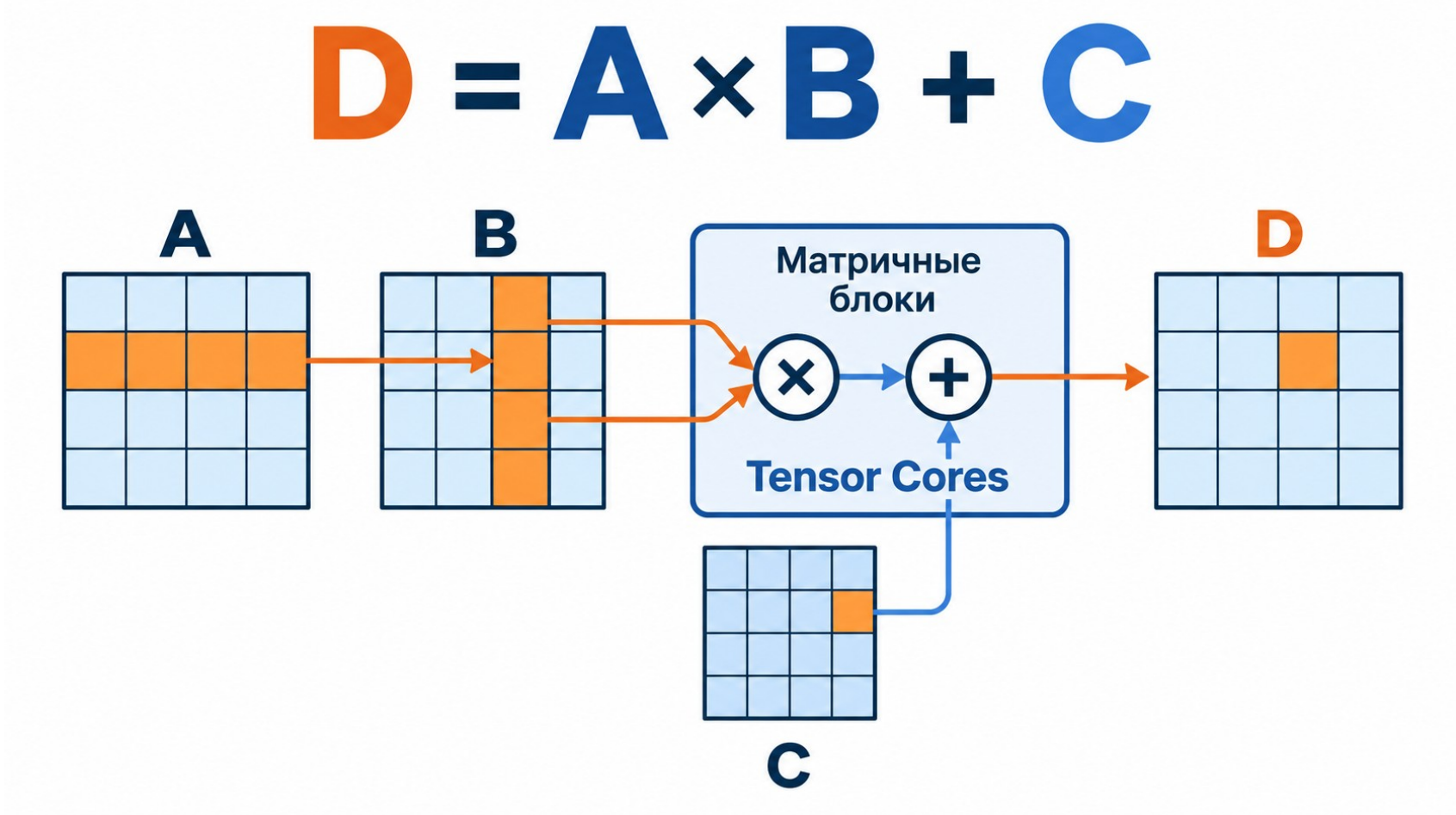
FP32, FP16, BF16 и INT8



Меньше бит — иной диапазон и точность представления.

PyTorch

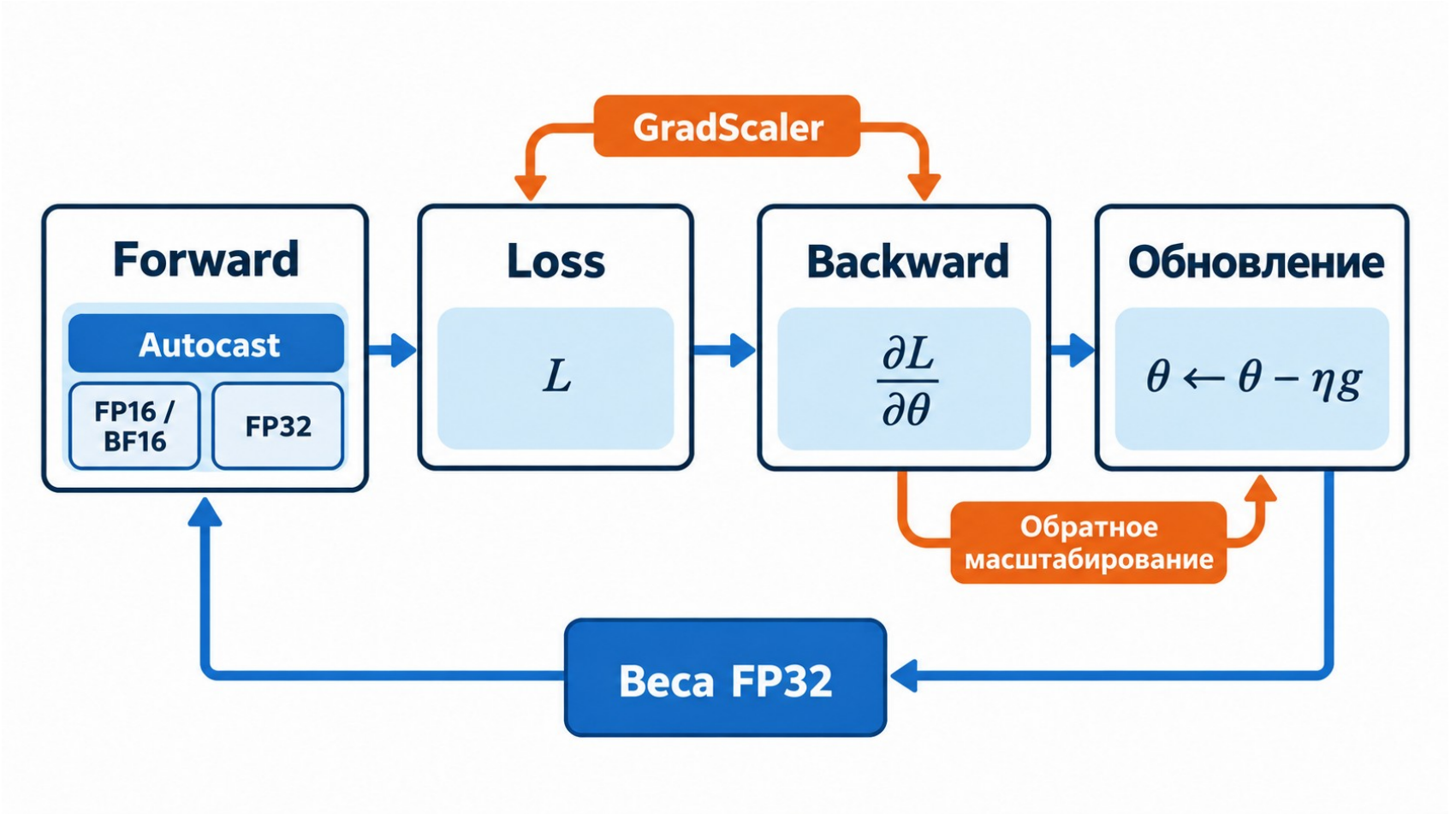
Матричные операции на Tensor Cores



Поддерживаемые форматы и размеры влияют на выбор реализации.

NVIDIA

Mixed precision и AMP



FP16: масштабируем loss до backward.

PyTorch

Как измерять время GPU

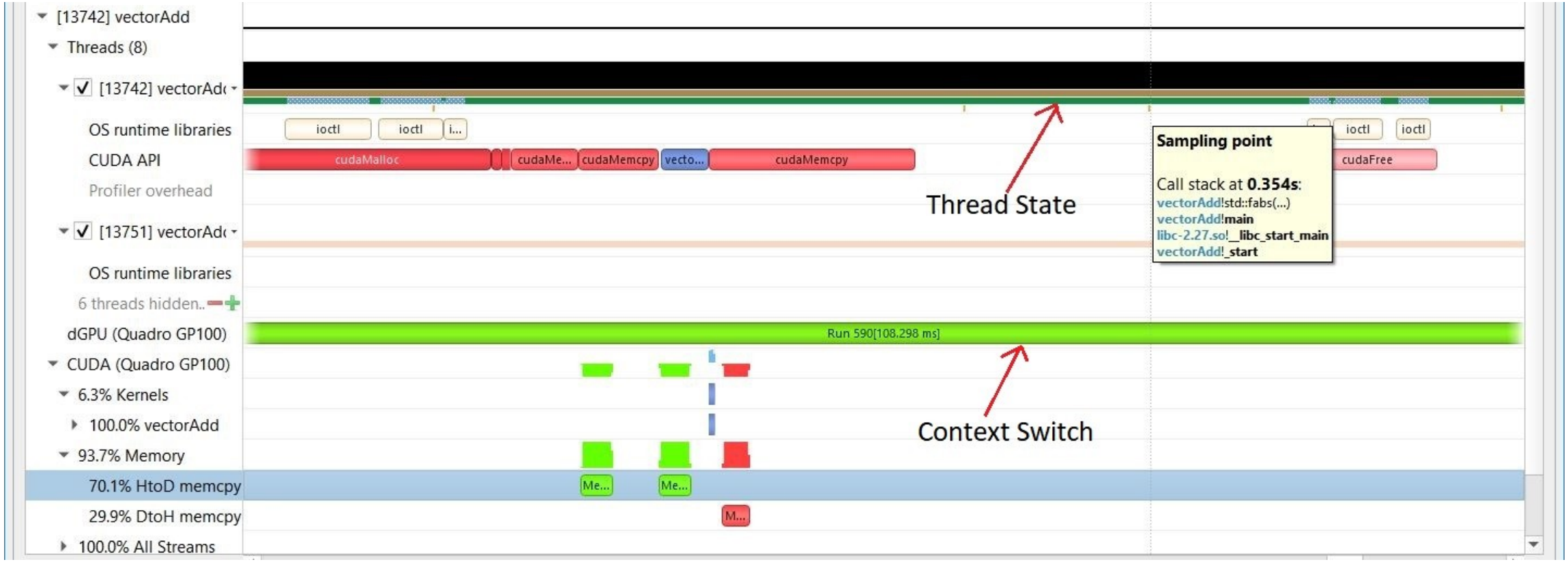
```
model.eval()
with torch.inference_mode():
    for _ in range(10): model(x)
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(30): model(x)
    end.record()
    end.synchronize()
    print(start.elapsed_time(end) / 30)   # ms/iter
```

Прогрев и синхронизация отделяют вычисление от постановки в очередь.

PyTorch

Читаем профиль выполнения

CPU / CUDA API **GPU: kernels и копирование**



Пробелы между GPU-операциями проверяем вместе с CPU и копированием.

[NVIDIA, оригинальный профиль]

Что показывает nvidia-smi

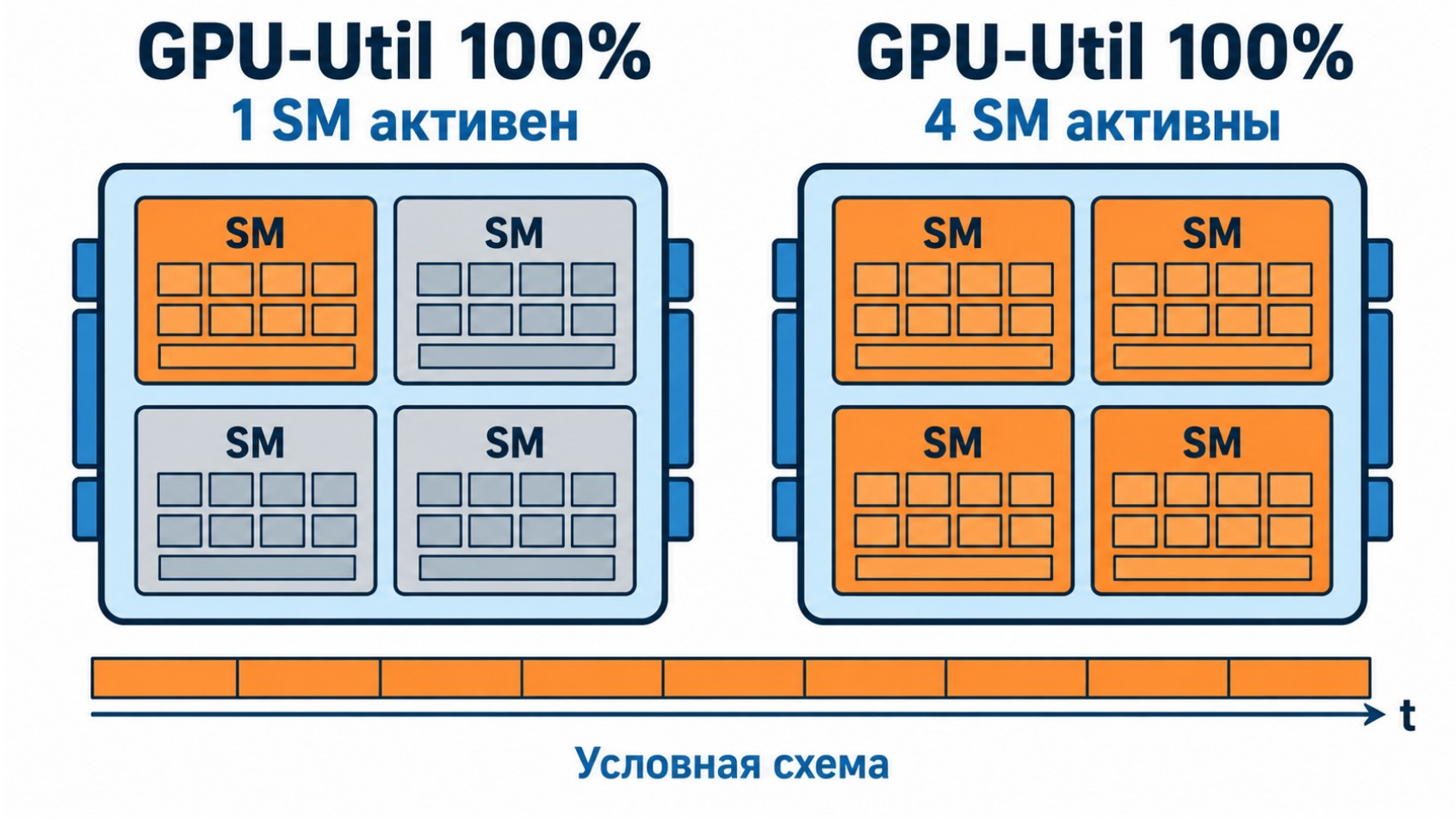
```
# Сбор временного ряда
nvidia-smi \
  --query-gpu=utilization.gpu,utilization.memory,memory.used \
  --format=csv -l 1

# Фрагмент опубликованного примера NVIDIA
# Memory-Usage          GPU-Util
# 0MiB / 15109MiB             0%
```

Memory-Usage — занятый объём. GPU-Util — время активности kernels.

NVIDIA

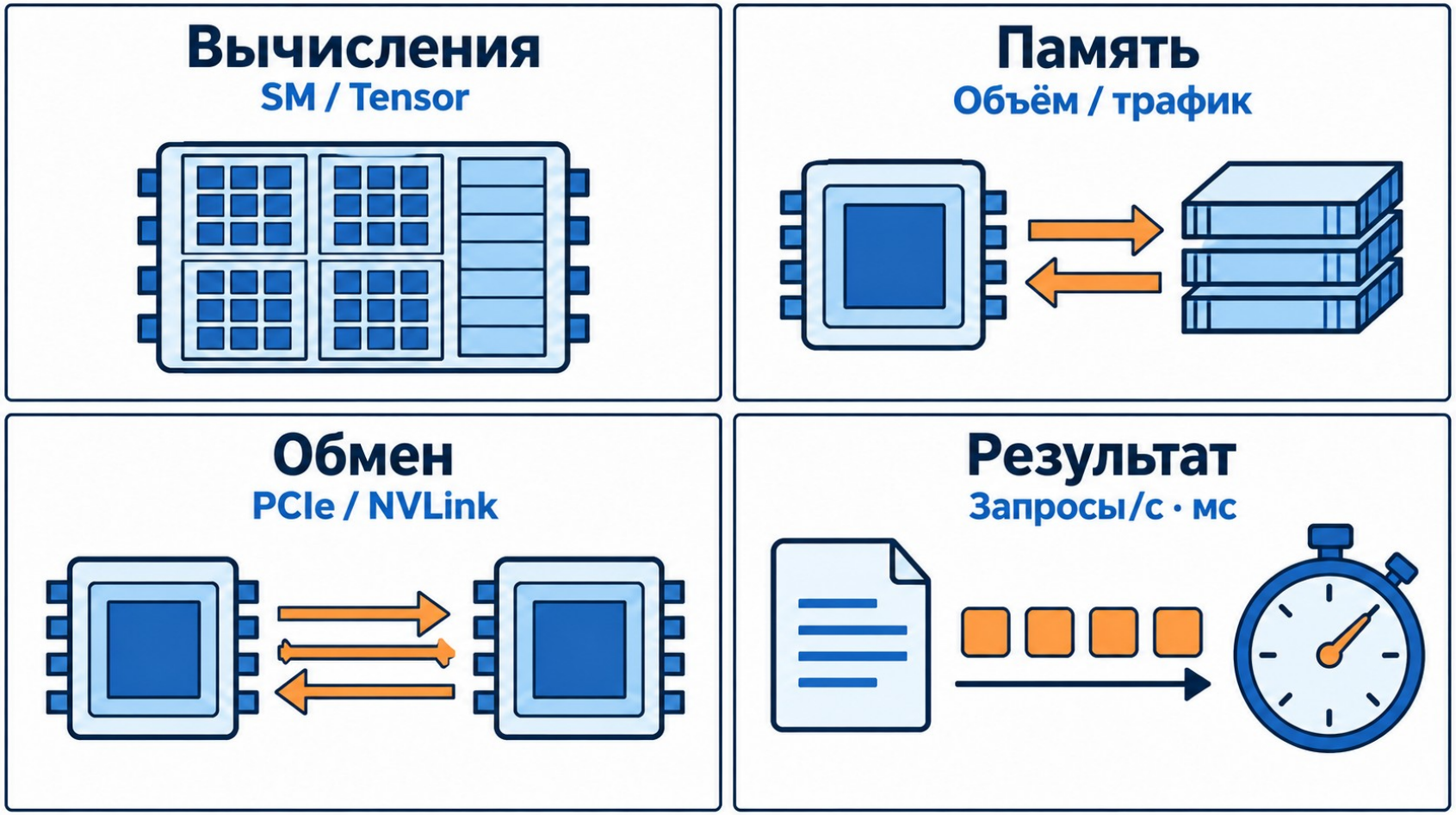
GPU-Util 100%: два разных случая



Окно одинаковое: SM activity 25% и 100%.

NVIDIA

Какие метрики нужны вместе



Сопоставляем MiB, GB/s, запросы/с и мс.

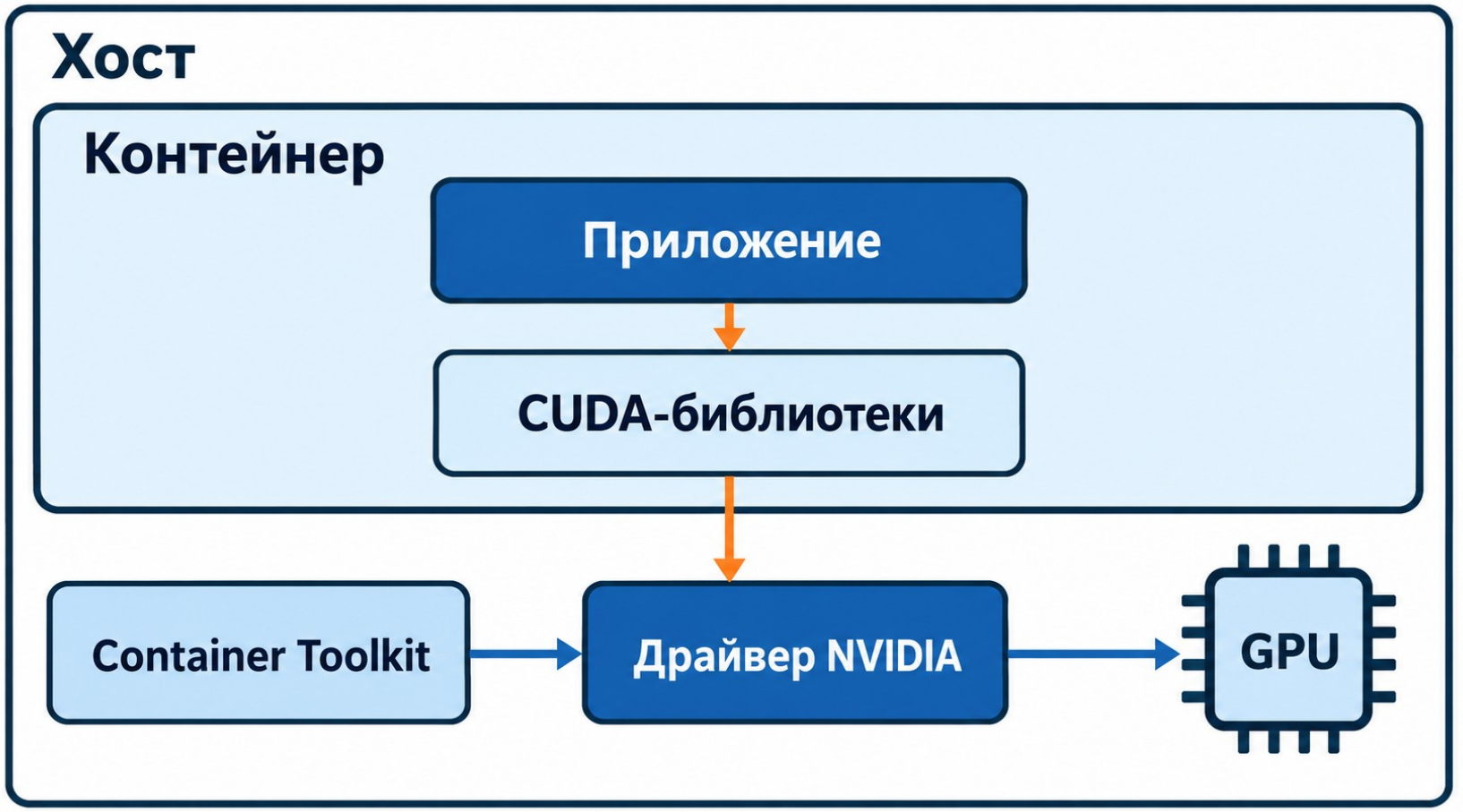
NVIDIA

От мониторинга к проверке причины



Гипотеза: мало параллельной работы. Проверка: размер запуска.

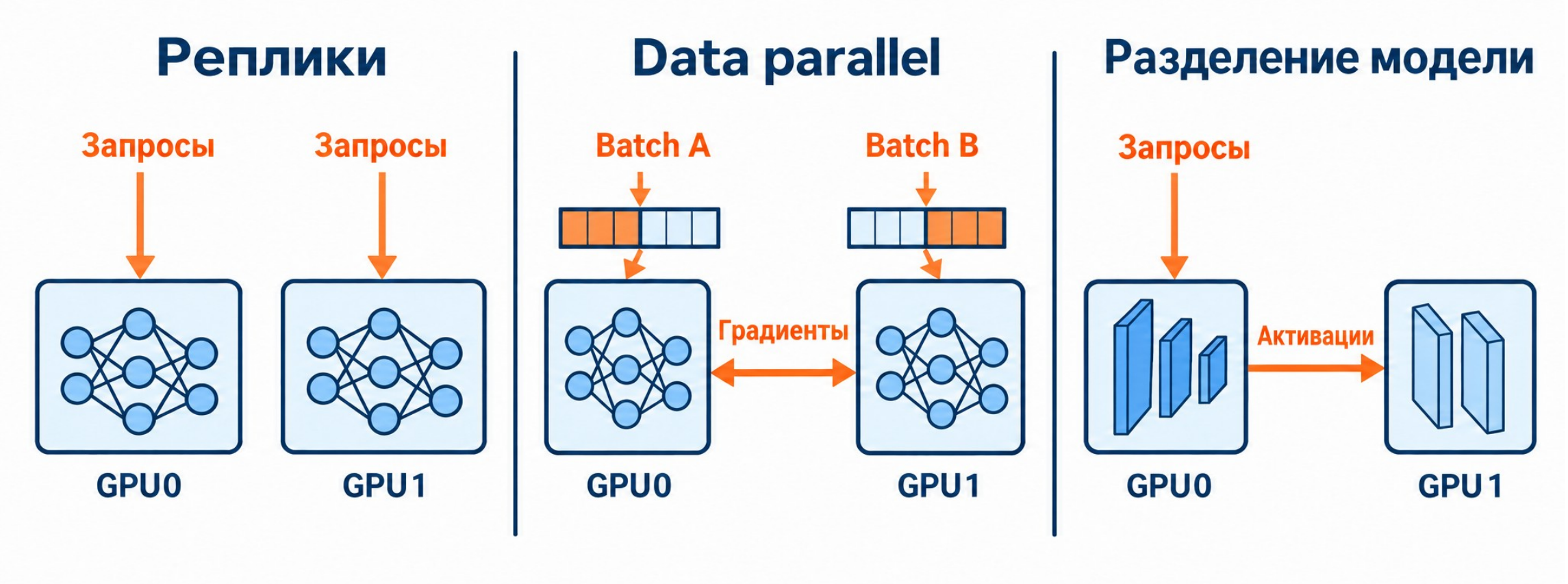
GPU внутри Docker



Драйвер остаётся на хосте; контейнеру предоставляется доступ к GPU.

NVIDIA

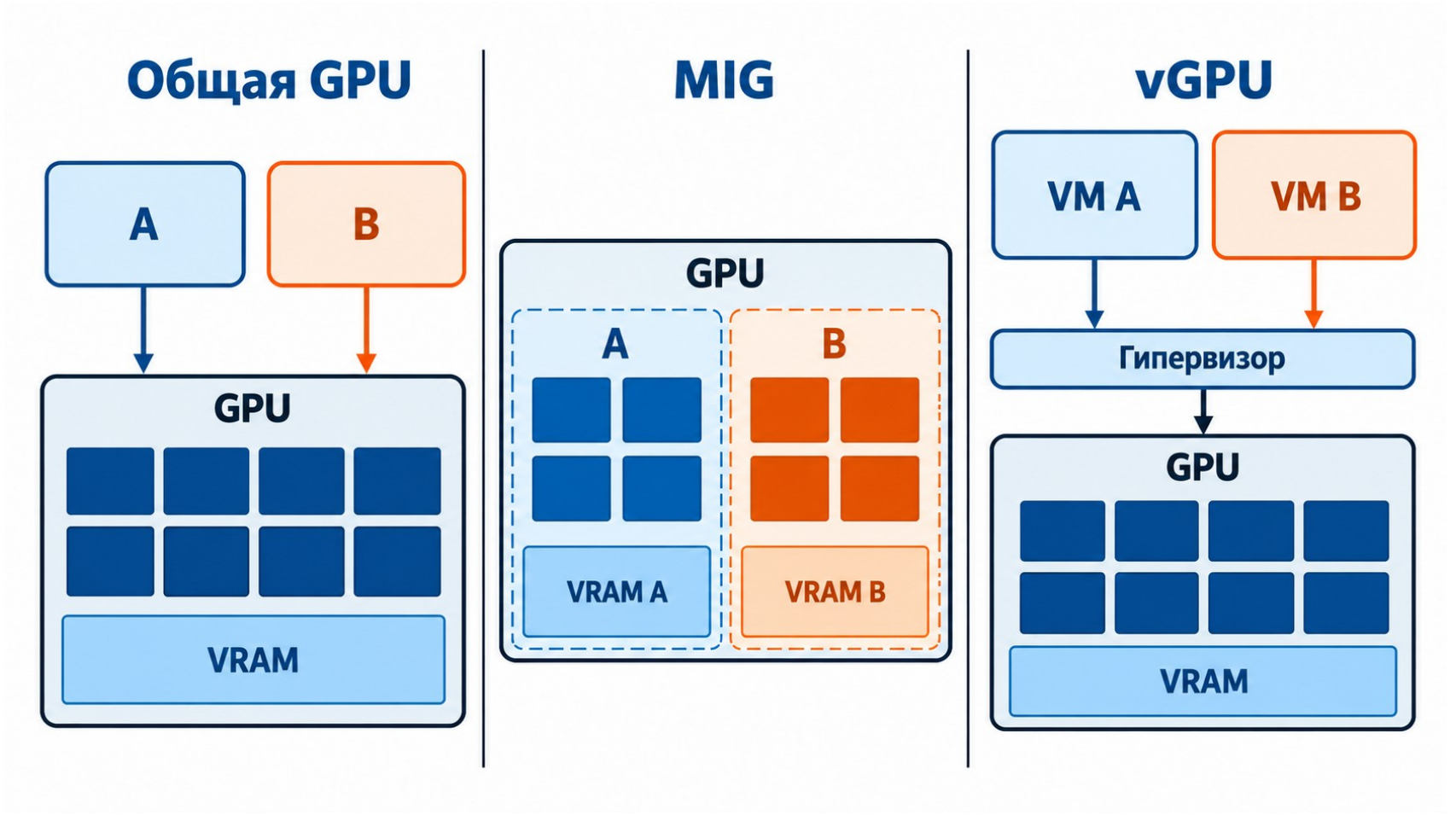
Несколько GPU: разные задачи



Разделяем работу между GPU.

PyTorch

MIG и vGPU



Распределяем GPU между пользователями.

NVIDIA

Итоги

- GPU выполняет массовую параллельную работу
- Передачи и память ограничивают скорость
- Точность меняет вычисления и расход памяти
- GPU-Util дополняем профилем и скоростью